

Name - Chaitanya Kelkar

Roll no - 15

Experiment No 3

Aim:

Create a Cryptocurrency (HadCoin) using Python and perform mining in a Blockchain implemented across a peer-to-peer network of multiple nodes.

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains transaction data, a timestamp, the previous block's hash, and its own unique hash. Once data is recorded, it becomes immutable, since altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of collecting pending transactions into a block, performing a computational Proof-of-Work puzzle to find a valid hash, and adding the new block to the chain. Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, three independent blockchain nodes were simulated on ports 5001, 5002, and 5003. Each node runs on a separate port, maintains its own copy of the blockchain, and can connect with peers to share and validate blocks.

4. Consensus Mechanism

The Longest Chain Rule was used: if multiple versions of the chain exist across nodes, the longest valid chain is chosen, ensuring all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction contains a sender, receiver, and amount. When a block is mined, all pending transactions are added to the block, and a reward transaction is automatically created to pay the miner. After mining, the transaction pool (mempool) is cleared.

6. Chain Replacement

When `/replace_chain` is called, a node requests chains from its peers; if a longer valid chain is found, it replaces its own — keeping the blockchain consistent across the network.

Output -

Fig 1: Three blockchain nodes running independently on ports 5001, 5002, 5003

```
node_5001.log X
C: > Users > CHAITANYA KELKAR > Downloads > Blockchain_Lab3 > node_5001.log
1  * Serving Flask app 'hadcoin_node_5001'
2  * Debug mode: on
3  WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
4  * Running on all addresses (0.0.0.0)
5  * Running on http://127.0.0.1:5001
6  * Running on http://192.168.0.100:5001
7  Press CTRL+C to quit
8  * Restarting with stat
9  * Debugger is active!
10 * Debugger PIN: 274-594-280
11 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /connect_node HTTP/1.1" 201 -
12 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /add_transaction HTTP/1.1" 201 -
13 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /add_transaction HTTP/1.1" 201 -
14 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /add_transaction HTTP/1.1" 201 -
15 127.0.0.1 - - [09/Sep/2026 01:19:46] "GET /mine_block HTTP/1.1" 200 -
16 127.0.0.1 - - [09/Sep/2026 01:19:46] "GET /get_chain HTTP/1.1" 200 -
17 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
18 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
19 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
20 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -

node_5002.log X
C: > Users > CHAITANYA KELKAR > Downloads > Blockchain_Lab3 > node_5002.log
1  * Serving Flask app 'hadcoin_node_5002'
2  * Debug mode: on
3  WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
4  * Running on all addresses (0.0.0.0)
5  * Running on http://127.0.0.1:5002
6  * Running on http://192.168.0.100:5002
7  Press CTRL+C to quit
8  * Restarting with stat
9  * Debugger is active!
10 * Debugger PIN: 819-301-442
11 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /connect_node HTTP/1.1" 201 -
12 127.0.0.1 - - [09/Sep/2026 01:19:46] "GET /get_chain HTTP/1.1" 200 -
13 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /replace_chain HTTP/1.1" 200 -
14 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
15 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
16 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -

node_5003.log X
C: > Users > CHAITANYA KELKAR > Downloads > Blockchain_Lab3 > node_5003.log
1  * Serving Flask app 'hadcoin_node_5003'
2  * Debug mode: on
3  WARNING: This is a production deployment. Do not use it in a production deployment. Use a production WSGI server instead.
4  * Running on all addresses (0.0.0.0)
5  * Running on http://127.0.0.1:5003
6  * Running on http://192.168.0.100:5003
7  Press CTRL+C to quit
8  * Restarting with stat
9  * Debugger is active!
10 * Debugger PIN: 541-628-109
11 127.0.0.1 - - [09/Sep/2026 01:19:46] "POST /connect_node HTTP/1.1" 201 -
12 127.0.0.1 - - [09/Sep/2026 01:19:46] "GET /get_chain HTTP/1.1" 200 -
13 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
14 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /replace_chain HTTP/1.1" 200 -
15 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
16 127.0.0.1 - - [09/Sep/2026 01:19:47] "GET /get_chain HTTP/1.1" 200 -
```

Fig 2: Peer-to-peer node connection — all three nodes registered with each other

```
POST http://127.0.0.1:5001/connect_node
Request: {"nodes":["http://127.0.0.1:5002","http://127.0.0.1:5003"]}
HTTP/1.1 201
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5003",
    "127.0.0.1:5002"
  ]
}
```

```

POST http://127.0.0.1:5002/connect_node
Request: {"nodes":["http://127.0.0.1:5001","http://127.0.0.1:5003"]}
HTTP 201
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5003",
    "127.0.0.1:5001"
  ]
}

POST http://127.0.0.1:5003/connect_node
Request: {"nodes":["http://127.0.0.1:5001","http://127.0.0.1:5002"]}
HTTP 201
{
  "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
  "total_nodes": [
    "127.0.0.1:5002",
    "127.0.0.1:5001"
  ]
}

```

Fig 3: Adding transactions to the mempool on node 5001

```

POST http://127.0.0.1:5001/add_transaction
Request: {"sender":"a","amount":51,"receiver":"b"}
HTTP 201
{
  "message": "This transaction will be added to Block 2"
}

POST http://127.0.0.1:5001/add_transaction
Request: {"sender":"a","amount":52,"receiver":"b"}
HTTP 201
{
  "message": "This transaction will be added to Block 2"
}

POST http://127.0.0.1:5001/add_transaction
Request: {"sender":"a","amount":53,"receiver":"b"}
HTTP 201
{
  "message": "This transaction will be added to Block 2"
}

```

Fig 4: Mining a block via Proof-of-Work, including miner reward transaction

```

GET http://127.0.0.1:5001/mine_block
HTTP 200
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "3ec40462312c03a07e5895b664940c39e92b54894a8df310159959c12560b833",
  "proof": 533,
  "timestamp": "2026-09-09 01:19:46.869099",
  "transactions": [
    {
      "amount": 51,
      "receiver": "b",
      "sender": "a"
    },
    {
      "amount": 52,
      "receiver": "b",
      "sender": "a"
    },
    {
      "amount": 53,
      "receiver": "b",
      "sender": "a"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "0325047fdfffb45a2ac28bb62f61676f7"
    }
  ]
}

```

Fig 5: Chain length mismatch across nodes before consensus (Condition 2)

```
GET http://127.0.0.1:5001/get_chain
HTTP 200
Chain length: 2

GET http://127.0.0.1:5002/get_chain
HTTP 200
Chain length: 1

GET http://127.0.0.1:5003/get_chain
HTTP 200
Chain length: 1
```

Fig 6: Consensus mechanism — nodes replacing their chain with the longest valid chain

```
GET http://127.0.0.1:5002/replace_chain
HTTP 200
{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-09 01:19:21.832577",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "3ec40462312c03a07e5895b664940c39e92b54894a8df310159959c12560b833",
      "proof": 533,
      "timestamp": "2026-09-09 01:19:46.869099",
      "transactions": [
        {
          "amount": 51,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 52,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 53,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "0325047fdffb45a2ac28bb62f61676f7"
        }
      ]
    }
  ]
}
```

```

GET http://127.0.0.1:5003/replace_chain
HTTP 200
{
  "message": "The nodes had different chains so the chain was replaced by the longest one.",
  "new_chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-09-09 01:19:21.832577",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "3ec40462312c03a07e5895b664940c39e92b54894a8df310159959c12560b833",
      "proof": 533,
      "timestamp": "2026-09-09 01:19:46.869099",
      "transactions": [
        {
          "amount": 51,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 52,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 53,
          "receiver": "b",
          "sender": "a"
        },
        {
          "amount": 1,
          "receiver": "Richard",
          "sender": "0325047fdffb45a2ac28bb62f61676f7"
        }
      ]
    }
  ]
}

```

Fig 7: Final synchronized blockchain state across all nodes

```

GET http://127.0.0.1:5001/get_chain
HTTP 200
Chain length: 2

```

```

GET http://127.0.0.1:5002/get_chain
HTTP 200
Chain length: 2

```

```

GET http://127.0.0.1:5003/get_chain
HTTP 200
Chain length: 2

```

Conclusion -

We implemented a cryptocurrency (HadCoin) in Python using Flask, simulating a three-node peer-to-peer blockchain network with Proof-of-Work mining and automatic miner rewards. Node 5001 mined a block, creating an intentional chain-length mismatch across nodes, which was resolved when nodes 5002 and 5003 synchronized via the Longest Chain Rule using /replace_chain. This demonstrated the core blockchain concepts of decentralized consensus, transaction validation, and network-wide synchronization.